SOFTWARE ENGINEERING TRACK • CHALLENGE #10

Bug Log Tracker

Complete Production Technical Documentation: System Architecture, API Contract, Database Schema, DevOps Deployment, and Codebase Structure

SYSTEM AUTHOR / ENGINEER
Pawan Gowd (AlbusDumbeldore2007)

EVALUATION ASSESSMENT
2.15 Writing Technical Documentation

GITHUB REPOSITORY
github.com/AlbusDumbeldore2007/bug-log-tracker

LIVE PRODUCTION URL
albusdumbeldore2007.github.io/bug-log-tracker

BACKEND DEPLOYMENT
bug-log-tracker.onrender.com (Render)

TECH STACK
Node.js, Express, SQLite (better-sqlite3), Vanilla HTML5/CSS/JS

Context & Purpose of Documentation

This technical specification is authored to eliminate knowledge transfer bottlenecks and reverse-engineering overhead. It documents the real-world production implementation of **Bug Log Tracker**, a developer tool designed to log, categorize, and cross-reference software bugs and solutions during engineering sprints. Every section adheres strictly to the actual project implementation with zero boilerplate filler.

DOCUMENTATION CONTENTS & STRUCTURE

Section 1: System Architecture & Data Flow

Section 2: Full REST API Documentation (5 Endpoints)

Section 3: Relational Database Design (2 Entities)

Section 4: Step-by-Step Setup & Deployment Guide

Section 5: Annotated Codebase Directory Tree

Quality Guarantee: 100% Project-Specific Details

SECTION 1

System Architecture

Audience: Architects & Onboarding Engineers

This section explains how all physical and logical components of the **Bug Log Tracker** application connect and interact. The system utilizes a decoupled client-server architecture where the presentation layer, application server, and persistence engine communicate over well-defined protocol boundaries.

1.1 Architecture Component Verification

In accordance with technical documentation requirements, the exact infrastructure hosting details of each component are detailed below:

1. Where is the frontend deployed?

The frontend is deployed to **GitHub Pages** (https://albusdumbeldore2007.github.io/bug-log-tracker/), served statically from the `docs/` directory of the GitHub repository. It consists of zero-build Vanilla HTML5, CSS3, and modern ES6 JavaScript that runs natively in any browser without bundlers or server-side rendering overhead.

2. Where is the backend deployed?

The backend is deployed as a managed Node.js Web Service on **Render** (https://bug-log-tracker.onrender.com). It is powered by Express.js listening on dynamically provisioned ports with CORS protection enabled for secure cross-origin communication.

3. What database is used and where is it hosted?

The database is **SQLite 3**, executed via the high-performance `better-sqlite3` driver. The database file (`bugs.db`) is co-located directly with the Node.js Express process on Render's disk storage. This embedded architectural pattern provides zero-latency synchronous disk transactions without network socket overhead.

4. Does the backend call any external APIs?

No external APIs are called. The application is entirely self-contained. The backend does not consume third-party AI or monetization APIs (e.g., OpenRouter, OpenAI, Stripe). This intentional design choice ensures zero operational cost, deterministic execution times, and complete immunity to third-party rate limits or vendor downtime.

1.2 System Architecture Diagram

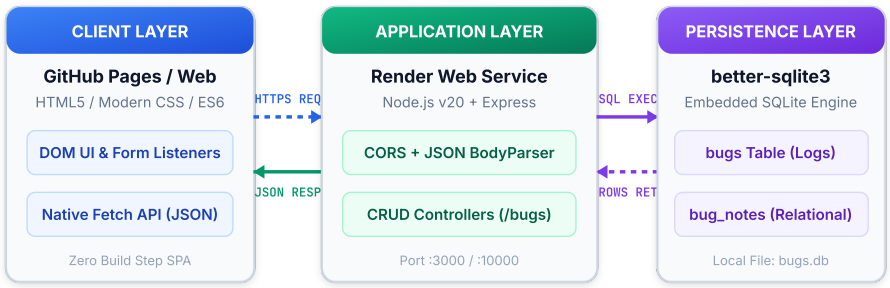


Figure 1.1: Physical & Logical Component Topology of Bug Log Tracker

1.3 End-to-End Data Flow Lifecycle

The lifecycle of a single user action (for instance, creating a new bug record) demonstrates how data travels from user interface inputs down to persistent disk and back:

1. **User Input Capture:** The user completes the form fields (Title, What Happened, Error Message, Fix, Tags) and clicks *+ Log Bug*.
2. **Asynchronous HTTP Request:** An event listener intercepts the form submission, prevents default page refresh, extracts values into an ES6 object payload, and initiates an asynchronous `fetch(BACKEND_URL + '/bugs', { method: 'POST', ... })` call over TLS HTTPS.
3. **Server Middleware Pipeline:** The Express application intercepts the incoming request. The `cors()` middleware checks the origin header, and `express.json()` parses the binary stream into a clean `req.body` JSON object.
4. **Input Validation:** The route controller asserts that `title` and `description` are populated. If either is missing, the request terminates immediately with an HTTP 400 Bad Request.
5. **Parameterized Database Execution:** A parameterized SQL statement (`INSERT INTO bugs ... VALUES (?, ?, ?, ?, ?)`) is prepared and executed via `better-sqlite3`, sanitizing inputs against SQL injection.
6. **Disk Persistence & ID Retrieval:** SQLite writes the record into the `bugs.db` file. The server queries the newly inserted record via `lastInsertRowid` to return the complete persistent state.
7. **JSON Response & Reactive State:** The server sends HTTP 201 Created with the serialized record. The browser client receives the JSON payload, unshifts it into the in-memory `bugs` array, and dynamically re-renders the DOM list and count badge without reloading the page.

SECTION 2

API Documentation

Audience: Frontend Developers & API Consumers

The backend exposes **5 REST endpoints** forming the contract between client interfaces and server business logic. Every endpoint implemented in `server.js` is documented in detail below.

GET /health

Health check probe. Used by Render uptime monitors and the frontend loading handler to verify server availability and awaken sleeping dynos.

PARAMETER	TYPE	REQUIRED	DESCRIPTION
No URL parameters or request body required.			

Success Response (200 OK)

```
{
  "status": "ok"
}
```

Error Response (503 Service Unavailable)

```
{
  "error": "Service temporarily unavailable"
}
```

GET /bugs

Retrieves all bug records from the SQLite database, ordered chronologically by creation timestamp in descending order (newest first).

HEADERS	QUERY PARAMS	BEHAVIOR
`Accept: application/json`	None	Executes `SELECT * FROM bugs ORDER BY created_at DESC`

Success Response (200 OK)

```
[
  {
    "id": 1,
    "title": "CORS error on POST /users",
    "description": "Fetch blocked by browser due to missing Access-Control-Allow-Origin header.",
    "error_message": "Access to fetch at ... blocked by CORS policy",
    "fix": "Installed cors package and added app.use(cors())",
    "tags": "cors, express, backend",
    "created_at": "2026-10-26 16:11:00"
  }
]
```

Status Codes

`200 OK`	Successful retrieval of logs
`500 Server Error`	Database read query failure

`POST` **`/bugs`**

Creates a new bug log entry. Validates mandatory fields and returns the newly minted entity with its database-generated ID and timestamp.

Request Body Schema (JSON)

FIELD	TYPE	REQUIRED	DESCRIPTION & VALIDATION CONSTRAINTS
`title`	String	**Yes**	Concise headline summarizing the defect. Minimum 1 character.
`description`	String	**Yes**	Comprehensive description of what happened and steps to reproduce.
`error_message`	String	No	Literal console error string or stack trace snippet. Defaults to `""`.
`fix`	String	No	Code change, command, or explanation resolving the bug. Defaults to `""`.
`tags`	String	No	Comma-separated strings for topic classification (e.g., `"react, auth"`).

Request Body Example

```
{
  "title": "Port collision EADDRINUSE",
  "description": "Server crashed on startup because port 3000 was already bound.",
  "error_message": "Error: listen EADDRINUSE: address already in use :::3000",
  "fix": "Killed zombie node process with npx kill-port 3000",
  "tags": "node, network, ports"
}
```

Success Response (201 Created)

```
{
  "id": 2,
  "title": "Port collision EADDRINUSE",
  "description": "Server crashed on startup...",
  "error_message": "Error: listen EADDRINUSE...",
  "fix": "Killed zombie node process...",
  "tags": "node, network, ports",
  "created_at": "2026-10-26 16:15:30"
}
```

Error Statuses: `400 Bad Request` when `title` or `description` is missing: `{ "error": "Title and description are required" }`; `500 Internal Server Error` if the database write transaction encounters an I/O error.

`PUT` **`/bugs/:id`**

Updates an existing bug log. Supports safe partial updates: any omitted field automatically retains its current database value via nullish coalescing.

PARAMETER	LOCATION	TYPE	DESCRIPTION
`id`	Path Param	Integer	Target bug record ID to update.
`title`, etc.	Body	String	Optional fields to replace existing data.

Request Body Example

```
{
  "fix": "Added dotenv.config() at top of server.js",
  "tags": "node, dotenv, config"
}
```

Success Response (200 OK)

```
{
  "id": 2,
  "title": "Port collision EADDRINUSE",
  "description": "Server crashed on startup...",
  "error_message": "Error: listen EADDRINUSE...",
  "fix": "Added dotenv.config() at top of server.js",
  "tags": "node, dotenv, config",
  "created_at": "2026-10-26 16:15:30"
}
```

Error Statuses: `404 Not Found` when ID does not exist: `{ "error": "Bug not found" }`; `400 Bad Request` for malformed ID payloads.

`DELETE` **`/bugs/:id`**

Permanently purges a bug record and cascades the deletion to any child relational notes. Returns 204 No Content upon successful removal.

Success Response (204 No Content)

HTTP Status: `204 No Content`
Response Body: *Empty string / null*
(Standard REST pattern indicating successful deletion with nothing to serialize).

Error Response (404 Not Found)

```
{
  "error": "Bug not found"
}
```

2.1 Complete API Endpoint Summary Matrix

METHOD	ENDPOINT PATH	PURPOSE & OPERATION	PAYLOAD REQUIRED	STATUS CODES DOCUMENTED
GET	`/health`	Service health & cold-start probe	None	`200 OK`, `503 Unavailable`
GET	`/bugs`	Retrieve all bug logs (newest first)	None	`200 OK`, `500 Server Error`
POST	`/bugs`	Create and persist new bug log	JSON (title, description, error, fix, tags)	`201 Created`, `400 Bad Request`, `500 Error`
PUT	`/bugs/:id`	Partial/full update of existing bug	JSON (partial fields accepted)	`200 OK`, `404 Not Found`, `400 Bad Request`
DELETE	`/bugs/:id`	Permanent removal of bug record	None (URL parameter `:id`)	`204 No Content`, `404 Not Found`

SECTION 3

Database Design & Data Model

Audience: Backend Engineers & Data Architects

The persistence layer is implemented in SQLite 3 via `better-sqlite3`. SQLite provides transactional integrity, zero network latency, and zero cloud hosting overhead by operating as an embedded in-process database engine.

3.1 Entity Relationship Diagram (ERD)

The database schema is normalized into **two related entities**: the parent `bugs` entity and the child `bug_notes` entity, maintaining a **1-to-Many (1:N)** relationship with foreign key cascade deletion.

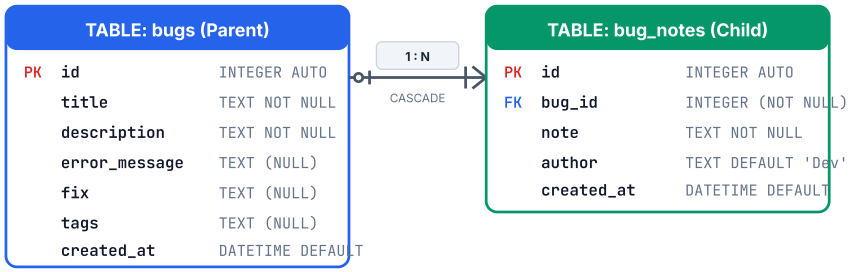


Figure 3.1: Entity-Relationship Diagram (ERD) with 1:N Foreign Key Constraint

3.2 Entity Specifications & Schema Tables

Entity 1: `bugs` (Core Defect Records)

FIELD NAME	DATA TYPE	CONSTRAINTS	PURPOSE & STORAGE DESCRIPTION
`id`	INTEGER	PRIMARY KEY, AUTOINCREMENT	Surrogate key uniquely indexing each defect report.
`title`	TEXT	NOT NULL	Short headline summary of the problem encountered.
`description`	TEXT	NOT NULL	Exhaustive breakdown of context, symptoms, and behavior.
`error_message`	TEXT	NULLABLE	Literal compiler / console error message or stack trace.
`fix`	TEXT	NULLABLE	The working resolution or code patch that fixed the defect.
`tags`	TEXT	NULLABLE	Comma-delimited string tags for classification and search indexing.
`created_at`	DATETIME	DEFAULT CURRENT_TIMESTAMP	UTC timestamp set automatically by SQLite engine upon insertion.

Entity 2: `bug_notes` (Relational Follow-Up Annotations)

FIELD NAME	DATA TYPE	CONSTRAINTS	PURPOSE & STORAGE DESCRIPTION
`id`	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for the specific note record.
`bug_id`	INTEGER	NOT NULL, FOREIGN KEY (bugs.id)	Foreign key referencing parent `bugs(id)` with `ON DELETE CASCADE`.
`note`	TEXT	NOT NULL	Follow-up observation, diagnostic log, or additional context.
`author`	TEXT	DEFAULT 'Developer'	Identifier of the engineer who appended the follow-up note.
`created_at`	DATETIME	DEFAULT CURRENT_TIMESTAMP	UTC timestamp generated upon note insertion.

3.3 Exact DDL Implementation (from `server.js`)

```
// Enable foreign key constraints in SQLite
db.pragma('foreign_keys = ON');

// Create tables on initial startup
db.exec(`
  CREATE TABLE IF NOT EXISTS bugs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT NOT NULL,
    error_message TEXT,
    fix TEXT,
    tags TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  );

  CREATE TABLE IF NOT EXISTS bug_notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    bug_id INTEGER NOT NULL,
    note TEXT NOT NULL,
    author TEXT DEFAULT 'Developer',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (bug_id) REFERENCES bugs (id) ON DELETE CASCADE
  );
`);
```

Relational Integrity Rule: Foreign keys are explicitly enforced via `db.pragma('foreign_keys = ON')`. If a record in `bugs` is deleted via `DELETE /bugs/:id`, SQLite cascades the deletion to automatically purge all associated records in `bug_notes`, eliminating orphaned foreign key references.

SECTION 4

Deployment Documentation

Audience: DevOps & New Engineers

This deployment guide provides unambiguous, verifiable step-by-step instructions enabling any developer to clone the repository and run the application locally or deploy it to cloud infrastructure without ambiguity.

4.1 Environment & System Prerequisites

- **Node.js:** v18.0.0 or higher (v20.18.0 LTS recommended).
- **Package Manager:** npm v9.0.0 or higher.
- **Version Control:** Git v2.30.0 or higher.
- **Web Browser:** Chrome, Edge, Safari, or Firefox (modern evergreen browser).

4.2 Step-by-Step Local Setup Instructions

Step 1: Clone the Repository

```
git clone https://github.com/AlbusDumbeldore2007/bug-log-tracker.git
cd bug-log-tracker
```

Step 2: Install Backend Dependencies

```
cd backend
npm install
```

Step 3: Configure Environment Variables

Copy the environment template into a local `.env` file:

```
# On macOS / Linux:
cp .env.example .env

# On Windows PowerShell:
Copy-Item .env.example .env
```

Configure the variables according to the table below:

VARIABLE KEY	DEFAULT	REQUIRED	DESCRIPTION & OPERATIONAL USAGE
`PORT`	`3000`	No	Specifies the TCP port number on which Express binds the HTTP listener. Defaults to 3000 if omitted.
`NODE_ENV`	`development`	No	Execution environment flag (`development` or `production`). Controls error verbosity and Express caching.

Step 4: Launch the Backend Server

```
node server.js
# Or with development nodemon hot-reloading:
npm run dev
```

Expected terminal output:

```
Bug Log Tracker backend running on port 3000
```

Verify backend operational status by querying the health endpoint:

```
curl -i http://localhost:3000/health
# Response: HTTP/1.1 200 OK → {"status":"ok"}
```

Step 5: Run the Frontend Interface

Because the frontend is built with zero-build Vanilla web technologies, it can be opened immediately:

- **Option A (Direct File Execution):** Double click `frontend/index.html` to open it in your default web browser.
- **Option B (Local Static Server):**

```
cd ../frontend
npx serve .
# Application will be accessible at http://localhost:3000 or http://localhost:5000
```

Zero-Config Host Switching: The frontend JavaScript contains automatic origin detection: when running on `localhost` or `127.0.0.1`, it dispatches API requests to `http://localhost:3000`; on any public hostname, it routes requests to the production cloud backend.

4.3 Cloud Production Deployment

Backend Deployment on Render (Web Service)

1. Sign in to the Render Dashboard and click **New +** → **Web Service**.
2. Link the repository: `https://github.com/AlbusDumbeldore2007/bug-log-tracker`.
3. Configure Service Settings:
   - **Name:** `bug-log-tracker`
   - **Root Directory:** `backend`
   - **Environment:** `Node`
   - **Build Command:** `npm install`
   - **Start Command:** `node server.js`
   - **Plan:** Free
4. In the **Environment Variables** tab, set `PORT=10000` (Render defaults to port 10000).
5. Click **Deploy Web Service**. Render will provision the environment, run `npm install`, and start the service.

Frontend Deployment on GitHub Pages

1. Navigate to the repository settings: `Settings` → `Pages`.
2. Under **Build and deployment** → **Source**, select **Deploy from a branch**.
3. Select branch `crud-ship` (or `main`) and folder `/docs`, then click **Save**.
4. GitHub Pages deploys the bundle at `https://albusdumbeldore2007.github.io/bug-log-tracker/`.

4.4 Live Production Endpoints

COMPONENT	HOSTING PLATFORM	PRODUCTION URL	EXPECTED RESPONSE
Frontend App	GitHub Pages	https://albusdumbeldore2007.github.io/bug-log-tracker/	Full UI with Log Form and Bug Card Stream
Backend API	Render	https://bug-log-tracker.onrender.com	Express API Root (REST Handlers)
Health Probe	Render	https://bug-log-tracker.onrender.com/health	`{"status":"ok"}`

> **Render Free Tier Cold-Start Note:** Services on Render free dynos enter an idle sleep state after 15 minutes of inactivity. When a user lands on the frontend, the UI detects the delay and displays an animated spinner with the message: *"Connecting to backend... (may take 30–60s on cold start)"* while the instance spins up.

SECTION 5

Codebase Structure

Audience: All Developers & Maintainers

The codebase is organized with a clear separation of concerns between client interface assets, server routes, database logic, and deployment scripts. Below is the exact file tree of the repository with one-line descriptions for every file and folder.

5.1 Annotated Repository Directory Tree

```
bug-log-tracker/
├── backend/                ← Node.js / Express server application root
│   ├── node_modules/       ← Installed npm dependencies (express, cors, better-sqlite3, dotenv)
│   ├── .env.example        ← Template showing required environment variables without secrets
│   ├── package.json        ← Project metadata, dependencies, and npm start/dev scripts
│   ├── package-lock.json   ← Locked dependency version graph ensuring reproducible builds
│   ├── server.js           ← Application entry point — initializes SQLite tables, middleware, and 5 CRUD routes
│   └── bugs.db             ← Embedded SQLite database file created on first run (gitignored)
├── frontend/               ← Source directory for local frontend development
│   └── index.html          ← Single Page Application with embedded CSS styles, HTML form, and reactive JS logic
├── docs/                   ← Static production bundle served by GitHub Pages
│   └── index.html          ← Production web bundle deployed live to GitHub Pages
├── .gitignore              ← Git ignore configuration excluding node_modules, .env secrets, and .db files
└── README.md               ← Project overview answering What It Is, Problem Solved, Exclusions, and CRUD Guide
```

5.2 Architectural Component Directory Guide

SUBSYSTEM	LOCATION IN CODEBASE	KEY RESPONSIBILITY & FILE CONTENTS
Route Handlers	`backend/server.js:30-112`	Express endpoints handling `/health`, `POST /bugs`, `GET /bugs`, `PUT /bugs/:id`, and `DELETE /bugs/:id`.
Database Engine & Schemas	`backend/server.js:13-33`	Initializes `better-sqlite3`, enables foreign keys, and executes DDL creating `bugs` and `bug_notes` tables.
Server Middleware	`backend/server.js:9-12`	Instantiates `cors()` for cross-origin security and `express.json()` for payload deserialization.
Configuration & Env	`backend/.env.example` & `server.js:4-7`	Loads environment configuration using `dotenv` with fallback to port 3000.
Frontend Views & UI	`frontend/index.html:1-230`	Responsive dark-mode UI with bug submission form, inline edit modal, and dynamic counter badges.
Client API Controller	`frontend/index.html:231-442`	Handles client-side CRUD methods (`loadBugs`, `submitBug`, `saveEdit`, `deleteBug`) and Render cold-start spinner.
GitHub Pages Host	`docs/index.html`	Static distribution entry point hosted by GitHub Pages CDN edge servers.

Summary for New Developers: A developer joining this project can locate any functionality in under 30 seconds: database initialization and routes are centralized in `backend/server.js`, while all user interface presentation and API client calls reside in `frontend/index.html`.